Multiplayer Pac-Man Engine in ARM7 Assembly

Author: Alex Rosillo Pino

Table of Contents

1. PROJECT INTRODUCTION AND MOTIVATION
2. HISTORY AND PARADIGM OF ASSEMBLY LANGUAGE
3. DEVELOPMENT ENVIRONMENT AND TARGET HARDWARE
4. ARCHITECTURAL DESIGN AND MEMORY MAPPING
5. INTERRUPT MANAGEMENT: KEYBOARD AND TIMER
6. SUBROUTINES AND STACK CONTEXT MANAGEMENT
7. MATHEMATICAL LOGIC AND BITWISE OPTIMIZATION
8. CONCLUSION

---

1. PROJECT INTRODUCTION AND MOTIVATION

Software development at the hardware level is an intensive exercise in logical structuring and architectural understanding. This project presents a fully functional, multiplayer implementation of the classic Pac-Man game, written entirely in **Assembly language** for the **ARM architecture**.

The primary motivation for choosing Assembly over high-level languages like C or C++ lies in the necessity of obtaining absolute control over the machine. Working in assembly eliminates the abstraction layers of modern compilers, forcing the developer to manage the processor flow, manipulate the stack directly, and communicate with hardware peripherals at the memory address level. This bare-metal approach drastically elevates logical reasoning and deepens the understanding of how modern computing inherently functions, assuming full responsibility for the data lifecycle.

2. HISTORY AND PARADIGM OF ASSEMBLY LANGUAGE

Assembly language emerged in the late 1940s as a human-readable representation of machine code. Before its invention, programmers had to enter binary or hexadecimal instructions directly. Assembly introduced symbolic opcodes (such as ADD, MOV, STR), revolutionizing programming speed and reducing human error when encoding logical sequences in the CPU.

Key advantages in this project:

- **Unmatched speed and resource efficiency:** The code is optimized at the clock cycle level, completely eliminating the overhead generated by compilers and achieving a real-time response crucial for a continuous-motion arcade game.
- **Direct hardware access:** It allows for the precise manipulation of specific memory addresses, general-purpose registers, and CPU status flags, enabling millimeter-accurate use of the available RAM.

Despite the steep learning curve and the complexity of debugging (lacking high-level constructs like while loops or object-oriented programming), mastering assembly remains the ultimate test of a software engineer's fundamental knowledge, as it requires thinking in the same way the machine executes tasks.

3. DEVELOPMENT ENVIRONMENT AND TARGET HARDWARE

The project was developed and verified using **Keil uVision 5**, an industry-standard and highly valued IDE for embedded systems. Keil provides a robust macro assembler and an environment simulator that replicates the physical behavior of integrated circuits with incredible precision. This simulator allows for direct observation of memory maps, real-time register states, and peripheral simulation (such as UART communication ports) without requiring interconnected physical hardware.

The target architecture is the **NXP LPC2105** microcontroller, built around the **ARM7TDMI-S** core. This 32-bit processor, featuring a Load/Store architecture, was selected for its rich set of integrated peripherals. In particular, the Vectored Interrupt Controller (VIC) and UART modules are intensively exploited, serving as the fundamental pillar for game synchronization and the asynchronous capture of commands from both players.

4. ARCHITECTURAL DESIGN AND MEMORY MAP

The program is structurally and semantically divided into the directives AREA datos, DATA and AREA codigo, CODE. In the data segment, memory is statically allocated for fundamental game state variables: remaining lives (vidas_j1, vidas_j2), scores, and the current positions of both the players and the ghost array.

The game board is conceptualized as a continuous 512-byte memory block (housed in the device's SRAM) that starts exactly at the base memory address 0x40007e00. Logically, this one-dimensional block is dynamically formatted as a two-dimensional grid of 32 columns by 16 rows. The rendering engine relies on ASCII values to determine graphical elements:

- . (ASCII 46): Consumable dots or "tokens" distributed across the map.
- @ (ASCII 64): Sprite corresponding to Player 1.
- & (ASCII 38): Sprite corresponding to Player 2.
- * (ASCII 42): Enemy entities (Ghosts) controlled in a pseudo-random manner.

- (ASCII 32): Blank space, indicating areas where tokens have already been consumed.

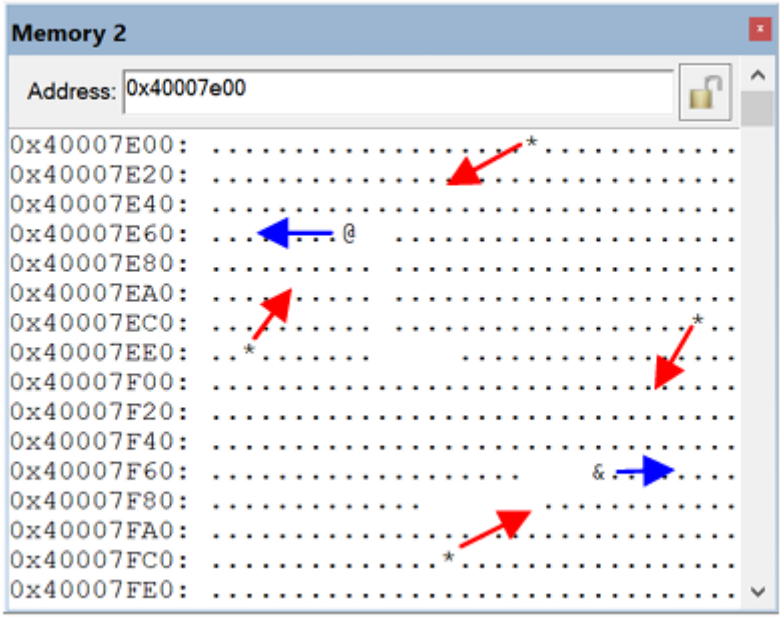

Figure 1: Interactive representation of the game board in the Keil simulator's 'Memory 2' window, showing the region starting from base address 0x40007E00. The sprites of the players and ghosts can be clearly seen navigating through the grid.

5. INTERRUPT MANAGEMENT: KEYBOARD AND TIMER

The implementation of pure interrupt service routines (ISR) in assembly is one of the greatest technical challenges overcome in this practice. When an external physical event occurs, the core interrupts the main thread. If the current context is not scrupulously saved, an irrecoverable corruption of the game logic will occur upon return.

Multiplayer Interface via UART

The UART #2 serial communication channel (linked to IRQ 7) acts as the controller's input interface. It allows simultaneous support for two players through asynchronous interrupt polling of terminal characters. When a key is pressed (such as 'W', 'A', 'S', 'D' or 'I', 'J', 'K', 'L'), the processor is interrupted, the RDAT register is read, and the direction vector is updated.

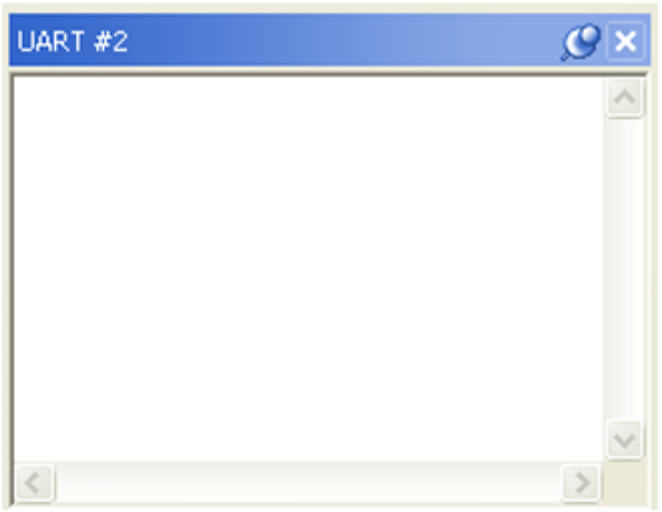

Figure 2: Keil's UART #2 serial terminal interface, used as the character capture port for directional movements.

Ultra-Low-Cost Computational ASCII Conversion

To ensure robust and case-insensitive control, an extremely optimized mechanism was implemented. Instead of executing expensive branches (if-else), the system evaluates whether the ASCII hexadecimal code falls within the [97, 122] range. If so, the "magic" instruction bic r0, r0, #2_100000 is executed. This bitwise operation (Bit Clear) "turns off" the fifth bit of the register, mathematically forcing the lowercase letter to mutate into uppercase in a single clock cycle. This is the essence of bare-metal optimization.

Dynamic Graphics Engine Control (Timer 0)

Rendering speed does not depend on free CPU cycles; instead, it is governed by hardware interrupts via Timer 0 (IRQ 4). The shared variable crono acts as a real-time clock. The game matrix only evaluates collisions and movements when this timer matches the next instant threshold. Additionally, the keyboard routine captures the '+' and '-' keys and intervenes in real-time on the max variable (using LSR and LSL logical shifts to divide/multiply by 2), accelerating or decelerating the physics of the entire environment without altering the collision logic.

6. SUBROUTINES AND STACK CONTEXT MANAGEMENT

The source code was structured using strong task encapsulation through subroutine labels (gen_personajes, procesar_tecla, act_pos_jug, etc.). Strictly adhering to the ARM architecture calling convention, each routine begins with state preservation via PUSH, which stores the

Link Register (LR) and all affected registers (r0-r7, r10) at the top of the system memory stack.

Before ending each routine, the original context is unpacked in symmetrically reverse order using POP, allowing the execution flow to be restored with precision (POP {..., pc}). In interrupt routines, this process is even more critical, as it requires safeguarding the Saved Program Status Registers (SPSR) by directly manipulating flags to return without execution context errors.

7. MATHEMATICAL LOGIC AND BITWISE OPTIMIZATION

When programming an embedded microcontroller that lacks specialized floating-point hardware and high-performance computational limiters, mathematical bottlenecks were bypassed using advanced Boolean engineering.

Pseudo-Random Mapping and Logical Masks

To provide the four ghosts with erratic artificial intelligence behavior, it was imperative to position them randomly. Usually, after invoking the imported C subroutine (rand), a mathematical modulo (%) would be applied to fit the 32-bit value within the board limits (512 bytes). Since divisions take multiple processor cycles in ARM assembly, the constant mask EQU 0x1FF was defined. This hexadecimal mask, equivalent to 511, is exactly 9 bits wide. By intersecting the random output using and r3, r3, r10, the resulting massive number is surgically pruned to always bounce between 0 and 511 in just 1 cycle, mapping directly into memory with ultra-efficiency.

Condition-Free Tunnel Behavior (Wrap-around)

One of the most sophisticated logical achievements of the program is the management of movement and board boundaries. The position update (act_pos_jug) decomposes a one-dimensional index (from 0 to 511) by extracting its two-dimensional X and Y vector properties:

- and r1, r1, #0x1F: Masks and isolates the column component (X Range: 0-31).
- bic r2, r1, #0x1F: Clears the previous mask, isolating purely the base row (Y Component).

This not only separates movements but automatically provides the natural behavior of "Pac-Man Tunnels." If the player crosses column 0 to the left, the arithmetic underflow would theoretically lead to negative indices, but when passed again through the #0x1F mask, the byte wraps around its result, forcing it to become the right boundary of the grid (31). This achieves instantaneous teleportation between screen edges without writing a single validation loop.

8. CONCLUSION

The design of this interactive engine in machine language not only validates a functional project but crystallizes a masterful demonstration of low-level engineering. Throughout the

source code, rigorous handling of asynchronous exceptions, direct manipulation of peripheral memory, ultra-high-speed arithmetic mask optimization, and precise stack cycle control are evident. Overcoming the inherent challenges of Assembly—such as direct I/O reading from serial terminals or on-the-fly binary restructuring without pre-compiled routines—confers invaluable analytical knowledge that bridges the gap between modern software abstraction and the physical reality of microprocessors.